

**The Problem**

Compute  $\tan(x)$  without any library performing the mathematics. No `math` module, no built-in factorial, no built-in trigonometry.

The constraint is what makes the work interesting: every decision a library would have made silently must now be made explicitly, and each becomes a requirement that can be verified.

**The Algorithm**

Maclaurin series for both components of the quotient identity:

$$\tan(x) = \frac{\sin(x)}{\cos(x)}$$
$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

**No factorial is ever computed.** Dividing one term by its predecessor gives the recurrence

$$t_{k+1} = t_k \cdot \frac{-x^2}{(2k)(2k+1)}$$

which costs one multiplication and one division per term and cannot overflow.

**Range reduction.** The tangent has period  $\pi$ , not  $2\pi$ , because `sin` and `cos` both change sign and the signs cancel in the ratio. Subtracting the nearest whole multiple of  $\pi$  confines the argument to  $[-\pi/2, \pi/2]$ , where the series converges quickly and every asymptote collapses onto a single check.

**Termination by accuracy, not by count.** Summation stops when a term falls below  $10^{-15}$  — the point at which it no longer changes the result at double precision.

**A Defect Found by Measurement**

Testing against a reference at  $x = 10^{15}$  returned  $-1.557$  where the true value is  $-1.672$  — wrong in the first decimal, with no error raised.

Sampling four thousand angles at each order of magnitude showed roughly one significant digit lost per factor of ten:

| Magnitude | Typical | Worst |
|-----------|---------|-------|
| $10^2$    | 13.6    | 10.8  |
| $10^4$    | 11.7    | 8.8   |
| $10^6$    | 9.7     | 6.8   |
| $10^8$    | 7.7     | 5.1   |

Results display ten decimal places. Past  $10^6$  the worst case falls below six correct digits, so the display would present noise as precision. **The validated range is  $\pm 10^6$** ; larger angles are refused with an explanation rather than answered.

All 636,620 exact asymptotes within that range were tested. None was missed.

**Unit Testing (PyUnit)**

**55 tests** across ten classes, each traced to a requirement from Deliverable 2. Six of the ten:

| Class                   | Requirement   |
|-------------------------|---------------|
| Reference angles        | TAN-FR-01, 03 |
| Range reduction         | TAN-FR-04     |
| Undefined points        | TAN-FR-07     |
| Validated range         | TAN-FR-11     |
| Termination             | TAN-FR-06     |
| From-scratch constraint | TAN-FR-02     |

**Validated by mutation.** Twenty defects were injected into the implementation; the suite detected seventeen. Of the three survivors, all were behaviour-preserving.

Two earlier survivors revealed genuine weaknesses — an unreachable validation branch masked by a later guard, and a magnitude boundary asserted relative to the constant it was meant to pin. Both are now covered.

**Quality Tooling**

```
(base) brandon@Konans-MacBook-Pro source_code % cd ~/Downloads/soen6011-tan-calculator/source_code
flake8 --max-line-length=79 tan_math.py tan_gui.py verify_tan.py && echo "Flake8: 0 issues"
pylint tan_math.py
pylint tan_gui.py
pylint verify_tan.py
Flake8: 0 issues

-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

(base) brandon@Konans-MacBook-Pro source_code %
```

|                           |            |
|---------------------------|------------|
| Flake8, 79-column limit   | no issues  |
| Pylint, all three modules | 10.00 / 10 |
| Unit tests                | 55 of 55   |
| Verification harness      | 44 of 44   |

**Debugger: Terms Shrinking to Tolerance**

```
((base) brandon@Konans-MacBook-Pro source_code % python -m pdb pdb_demo.py
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/pdb_demo.py(1)<module>()
-> """Driver for the pdb demonstration."""
(Pdb) break tan_math.py:242
Breakpoint 1 at /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py:242
(Pdb) continue
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(242)sine()
-> term == -x * x / ((2.0 * k) * (2.0 * k + 1.0))
(Pdb) p k, term, total
(1, 0.9735361584457678, 0.9735361584457678)
(Pdb) continue
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(242)sine()
-> term == -x * x / ((2.0 * k) * (2.0 * k + 1.0))
(Pdb) p k, term, total
(2, -0.1537818244191063, 0.8197543340266615)
(Pdb) continue
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(242)sine()
-> term == -x * x / ((2.0 * k) * (2.0 * k + 1.0))
(Pdb) p k, term, total
(3, 0.007287510376427245, 0.8270418444030808)
(Pdb) continue
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(242)sine()
-> term == -x * x / ((2.0 * k) * (2.0 * k + 1.0))
(Pdb) p k, term, total
(4, -0.0001644500722499108, 0.826877394330839)
(Pdb) p abs(term) < TOLERANCE
False
(Pdb) quit
(base) brandon@Konans-MacBook-Pro source_code %
```

Successive terms fall  $0.97 \rightarrow 0.15 \rightarrow 0.007 \rightarrow 0.0001$ . The stopping rule is visible rather than asserted.

**What the Faults Had in Common**

Four defects surfaced: silent precision loss at large arguments, a signature mismatch between the modules, a documented command that failed on a fresh clone, and an unreachable validation branch.

**Every one appeared only when something was run. None was visible by reading.**

**Which UI Principles Apply?**

```
graph LR
    A((UI Principles)) --- B((Help and docs))
    A --- C((Does not apply (2)))
    A --- D((Closure))
    A --- E((Reduced form (5)))
    A --- F((Recognition))
    A --- G((User controls))
    A --- H((Applies (11)))
    A --- I((Error diagnosis))
    A --- J((System status))
    A --- K((Error prevention))
```

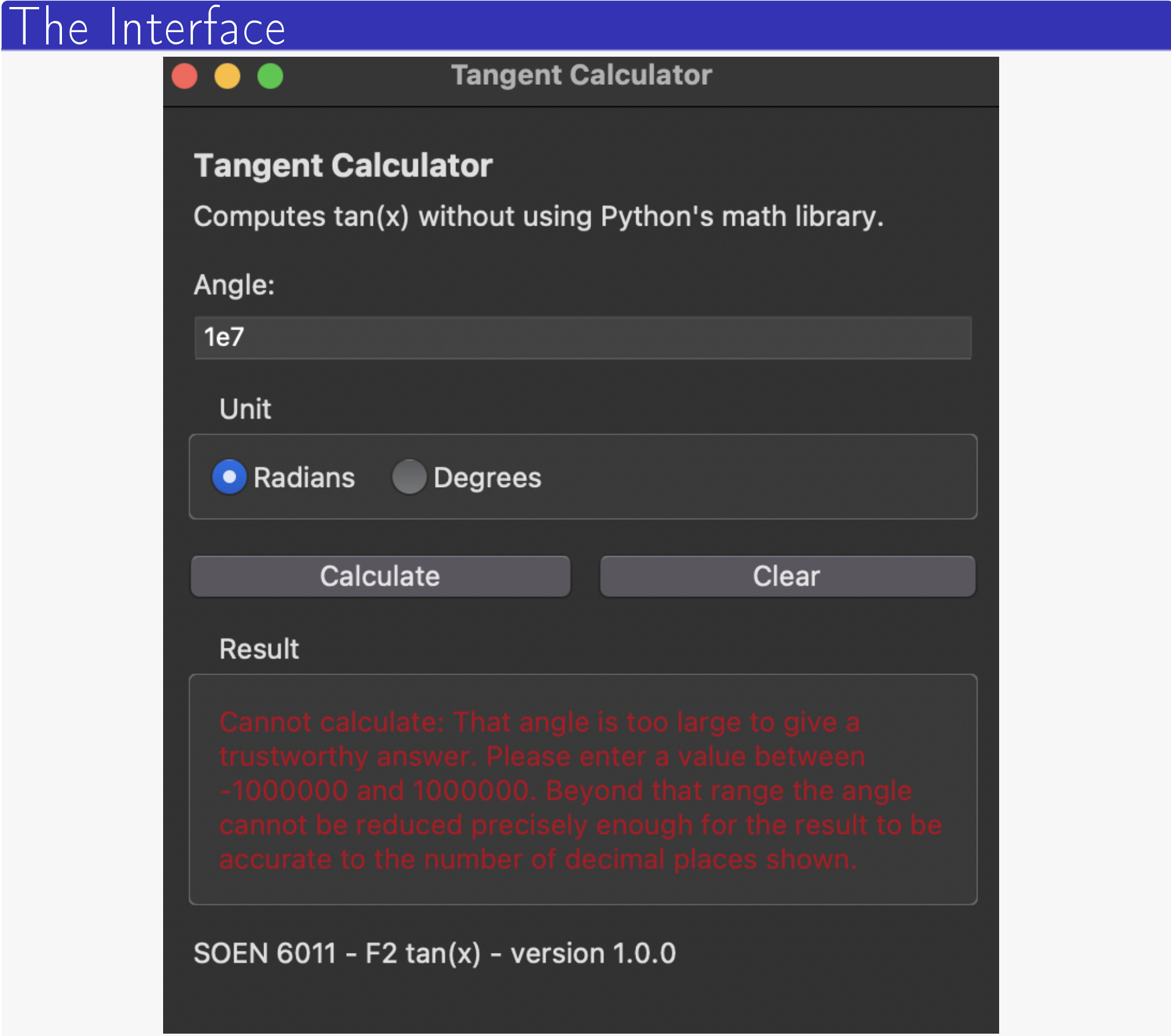
Nielsen's heuristics and Shneiderman's rules were assessed individually. Two do not apply: a **help system** for six labelled controls would be evidence the labels failed, and **closure** presupposes a sequence an atomic task lacks.

**Accessibility**

The interface aims to be accessible at the level the framework permits:

- Every control carries a visible text label
- All functionality reachable from the keyboard; Return calculates, Escape clears
- Focus begins in the entry field and is visibly indicated
- Every outcome is stated in words as well as colour**, so nothing depends on colour perception
- Contrast measured at 6.9:1 and 7.1:1 against the 4.5:1 requirement
- Text wraps rather than clipping; system font settings apply

**Stated limitation.** Screen-reader conformance is not claimed: Tkinter exposes no accessibility tree on any platform, so WCAG 4.1.2 and 4.1.3 cannot be met within this framework. That is a property of Tkinter, and is recorded rather than worked around.



An angle beyond the validated range, refused with an explanation rather than a fault code.

**Generative AI Use**

Four CASTROFF-structured prompts informed this deliverable: a chain-of-thought analysis of which UI principles apply, an accessibility audit against WCAG 2.1, a test-suite review, and a debugger session design.

The test review found three real weaknesses — a vacuous assertion, an unreachable branch, and a boundary asserted relative to the constant it was meant to pin. All are now covered. Prompts and responses are in [screenshots/](#).

**Repository**

github.com/Brand0-code/  
soen6011-tan-calculator

Version v1.0.0 — Semantic Versioning